

## C++ Exercise : 16. 모던 C++ 확장 기능과 최적화 (Set 2)

1. 컴파일러가 함수가 반환하는 임시 객체를 복사하거나 이동하는 대신, 호출자의 메모리 공간에 직접 객체를 생성하여 복사 비용을 완전히 없애는 최적화를 무엇이라고 하는가? (하)

1. 복사 생략 (Copy Elision / RVO)
2. 인라인 확장 (Inline Expansion)
3. 지연 평가 (Lazy Evaluation)
4. 타입 소거 (Type Erasure)

2. 함수의 반환값을 호출자가 무시할 경우 컴파일러가 경고(Warning)를 발생시키도록 유도하는 C++17 표준 어트리뷰트(Attribute)는 무엇인가? (하)

1. `[[maybe_unused]]`
2. `[[nodiscard]]`
3. `[[fallthrough]]`
4. `[[deprecated]]`

3. switch-case 문에서 break 문을 생략하고 의도적으로 다음 case 블록으로 흐름이 이어짐을 컴파일러와 다른 개발자에게 명시하여 경고를 방지하는 어트리뷰트는 무엇인가? (중)

1. `[[fallthrough]]`
2. `[[nodiscard]]`
3. `[[maybe_unused]]`
4. `[[noreturn]]`

4. 사용하지 않는 변수나 매개변수로 인해 발생하는 컴파일러의 경고를 억제하기 위해 선언부에 붙이는 어트리뷰트는 무엇인가? (하)

1. `[[nodiscard]]`
2. `[[fallthrough]]`
3. `[[maybe_unused]]`
4. `[[noexcept]]`

5. `std::variant`에 저장된 현재 타입에 따라 알맞은 함수나 연산을 안전하게 실행(방문)하기 위해 사용하는 표준 라이브러리 함수는 무엇인가? (상)

1. `std::invoke`
2. `std::visit`
3. `std::apply`
4. `std::any_cast`

6. `std::optional`에 값이 들어있지 않은 상태(Empty)에서 `operator*`를 통해 값에 접근할 때 발생하는 결과로 올바른 것은? (중)

1. 기본값(0 또는 빈 문자열)이 반환된다.
2. `std::nullopt` 예외를 던진다.
3. 미정의 동작(Undefined Behavior)이 발생한다.
4. 자동으로 새로운 객체를 기본 생성한다.

7. 힙(Heap) 메모리 할당 없이, 연속된 메모리 공간을 가지는 배열이나 `std::vector` 등의 데이터를 소유권 없이 안전하게 가리키 수 있는 C++20의 경량 뷰(View) 타입은 무엇인가? (중)

1. `std::string_view`
2. `std::span`
3. `std::optional`
4. `std::reference_wrapper`

8. 문자열 리터럴이나 기존 문자열을 복사하지 않고, 시작 주소와 길이를 통해 읽기 전용으로 안전하게 참조하기 위한 C++17 표준 타입은 무엇인가? (하)

1. `std::span`
2. `std::string_view`
3. `std::variant`
4. `std::vector_view`

9. C++20에서 조건에 따라 생성자의 암시적 형 변환(Implicit Conversion) 여부를 동적으로 제어할 수 있게 도입된 문법은 무엇인가? (상)

1. `explicit(bool)`
2. `virtual(bool)`
3. `constexpr(bool)`
4. `noexcept(bool)`

10. 람다 캡처 시 현재 객체의 멤버변수나 함수에 접근할 때, 객체 전체를 값(복사) 방식으로 캡처하여 원본 객체가 소멸한 뒤에도 안전하게 참조할 수 있게 해주는 C++20 문법은 무엇인가? (상)

1. `[this]`
2. `[*this]`
3. `[=this]`
4. `[&this]`

11. 프로그램 전체의 실행 프로파일(실행 빈도 데이터)을 수집하고, 이를 바탕으로 컴파일러가 자주 실행되는 분기를 최적화하도록 지시하는 컴파일 최적화 기법은 무엇인가? (중)

1. LTO (Link-Time Optimization)

2. PGO (Profile-Guided Optimization)
3. RVO (Return Value Optimization)
4. DOD (Data-Oriented Design)

12. 여러 번역 단위(소스 파일)로 나누어진 오브젝트 파일들을 모아, 링크 단계에서 전체 소스 코드를 대상으로 인라인 확장 등 전역적인 최적화를 수행하는 기술은 무엇인가? (중)

1. PGO (Profile-Guided Optimization)
2. LTO (Link-Time Optimization)
3. RAII (Resource Acquisition Is Initialization)
4. SFINAE

13. 메모리 접근 속도를 높이기 위해 CPU 캐시 적중률(Cache Hit Rate)을 극대화하도록, 데이터를 객체 단위가 아니라 연속된 배열 구조로 배치하는 설계 철학을 무엇이라고 하는가? (중)

1. 객체지향 설계 (OOD)
2. 데이터 지향 설계 (Data-Oriented Design, DOD)
3. 디자인 패턴 (Design Patterns)
4. 제네릭 프로그래밍 (Generic Programming)

14. C++20에서 비동기 프로그래밍 시 함수 실행을 일시 중단하고 제어권을 호출자에게 반환할 때 사용하는 키워드는 무엇인가? (중)

1. `co_return`
2. `co_yield`
3. `co_await`
4. `co_suspend`

15. C++20에서 비동기 코루틴 안에서 값을 하나씩 순차적으로 생성하여 반환(방출)할 때 사용하는 키워드는 무엇인가? (상)

1. `co_return`
2. `co_yield`
3. `co_await`
4. `co_send`

16. C++20에서 기존 `reinterpret_cast`가 가진 엄격한 타입 규칙 위반 문제를 해결하고, 컴파일 시점에 객체의 비트 패턴을 다른 타입으로 안전하게 재해석할 수 있게 도입된 함수는 무엇인가? (상)

1. `std::bit_cast`
2. `std::launder`
3. `std::invoke`
4. `std::any_cast`

17. 객체의 수명이 끝난 메모리 위치에 동일한 타입의 새로운 객체가 생성되었을 때, 컴파일러의 최적화로 인해 기존 포인터가 잘못 최적화되는 것을 방지하기 위해 사용하는 표준 함수는 무엇인가? (상)

1. `std::bit_cast`
2. `std::launder`
3. `std::forward`
4. `std::move`

18. 함수 포인터, 멤버 함수 포인터, 함수 객체 등 호출 가능한 모든 대상을 일관된 문법으로 호출할 수 있게 해주는 표준 유틸리티 함수는 무엇인가? (중)

1. `std::invoke`
2. `std::apply`
3. `std::forward`
4. `std::bind`

19. 튜플(Tuple)이나 인자의 집합을 풀어헤쳐 함수의 개별 인자들로 전달하며 호출할 때 사용하는 표준 유틸리티 함수는 무엇인가? (상)

1. `std::invoke`
2. `std::apply`
3. `std::forward`
4. `std::visit`

20. C++20 지정 초기화자(Designated Initializers)를 사용할 때 반드시 지켜야 하는 문법적 제약 사항으로 가장 올바른 것은 무엇인가? (중)

1. 선언된 멤버 변수의 순서와 완전히 동일하게 지정해야 한다.
2. **private** 멤버 변수만 지정할 수 있다.
3. 포인터 타입 멤버 변수에는 적용할 수 없다.
4. 반드시 힙에 생성되는 객체에만 사용할 수 있다.